

Robotic Photographer for 3D Gaussian Splatting

Team Splattoon

Arnav Narain, Charles Ciampa, Jason Cheung, Joseph Gu

April 2025

1 Introduction

Gaussian Splatting is an emerging method in 3D reconstruction to leverage point-based rendering to create highly detailed 3D representations from multi-angle images of an object. Introduced in the 2023 paper *3D Gaussian Splatting for Real-Time Radiance Field Rendering* by Kerbl et al. [2], the technique represents scenes using a collection of 3D Gaussians instead of traditional mesh or voxel-based structures.

Our team set out to design and build a robotic system capable of capturing the necessary multi-view image data to generate high-quality 3D constructions. Specifically, we aimed to develop a robot that could autonomously detect and move around an object, capturing a diverse set of images from all angles via the camera mounted on top of the robot. As a desirable feature, our group intended on using the robotic arm to capture a top-down image of the object with a second camera. These images would then be processed using Gaussian Splatting to produce a complete and photorealistic 3D model of the object.

The challenge of this task comes from the need for precise control and planning in both robot locomotion and arm articulation. The robot would have to detect the primary object in a scene without April Tags, and travel towards its centroid. Then, it would need to compute an optimal radius to orbit the object without collisions. Additionally, the robot arm would have to navigate a tight and unknown region without colliding with the object to capture a top-down photo. The final images would have to be high-quality and comprehensive so that the reconstruction could fully generate the 3D model.

Perception and planning are both capabilities that this project will tackle. The 3D reconstruction via Gaussian Splatting is a form of perception, and the robot will also need to perceive the centroid of the object to perform a trajectory around it. The robot will need to plan this trajectory such that the orbit has sufficient radius to not collide with the object/environment, enables accurate camera positioning to capture the object, and for the arm (an optional future feature) to maneuver over the top/interior of the object.

2 Implementation

Our implementation consisted of five primary stages: initialization, object identification, approach, object circumnavigation and scanning, and finally, object reconstruction. The robot employed for this task was the LoCoBot WX250 6DOF arm mounted on a Create 3 mobile base, utilizing ROS 1 Noetic on Ubuntu 20.04. Communication between the robot and a remote server was handled over an iPhone hotspot operating in the 2.4GHz range, using ZeroMQ sockets for efficient and reliable data transfer.

2.1 Initialization

Initialization began with launching the remote server on a laptop. The server leverages ZeroMQ for networking with the robot for images. It is built from a fork of the codebase for FreeSplat [3], a deep learning model for predicting 3D Gaussians from input images. We detail at the end of this section our reasons for selecting FreeSplat. The server also utilizes Splatviz [1] for the renderer window to interactively view the 3D result. After processing, Splatviz automatically launches to visualize the generated 3D reconstruction. Simultaneously, we ssh into the robot and launch the robot’s run script when the server is ready.

2.2 Object Identification

Initially, the robot oriented its camera forward to capture an RGB-D image without any preprocessing. The RGB image component underwent segmentation using the Python library and model `rembg`, effectively masking out the background. This mask was then applied to the corresponding depth image. The masked coordinates and the masked depth values were averaged and used to estimate the object’s 3D position in the camera’s frame of reference. We use the camera intrinsics here to convert from screen coordinates to real world coordinates. The depth axis coordinate was further adjusted by an experimentally determined calibration delta to improve accuracy.

We also experimented with YOLO object detection to accomplish this phase of the project, but simply went with `rembg` as this approach began working first.

2.3 Approaching the Object

The robot determined the required rotation angle to face the detected object using NumPy’s `arctan2` function within the `get_dist_and_rot()` function. This calculation involved converting object position from camera space to world space and accounted for coordinate transformations and camera tilt via a rotation matrix. Once calculated, the robot executed rotation by publishing velocity commands using ROS `geometry_msgs/Twist` messages to the topic `/mobile_base/cmd_vel`. Angular and linear velocities were precisely controlled, facilitating rotation towards the object followed by forward movement to reach a specified approach radius of approximately 1 meter.

2.4 Circumnavigation and Object Scanning

At approximately 0.75 meter from the object, the robot executed a 90-degree rotation to achieve a perpendicular orientation relative to the object. The camera was then rotated to maintain focus on the object throughout scanning. The robot followed a circular trajectory with a radius of 0.75 meters, moving at an angular velocity of about 0.5 radians per second (approximately 28.6 degrees per second), corresponding to a linear velocity of approximately 0.375 m/s. Every 3 seconds during this circumnavigation, the robot swiveled towards the object (to better aim the camera at the object) and paused. Each pause lasted 2 seconds to ensure image stabilization, after which an RGB-D image was captured. Images were encoded into PNG format using OpenCV’s `imencode()` function and transmitted to the remote server via ZeroMQ. Upon receipt, the server decoded and saved these images both in memory and to disk, subsequently sending a confirmation back to the robot. After completing roughly one-third of the circumnavigation—resulting in four captured images—the robot terminated the circumnavigation procedure.

2.5 Object Reconstruction

The remote server used the FreeSplatler [3] codebase for 3D Gaussian Splatting reconstruction, specifically employing the first and last images captured during the circumnavigation. FreeSplatler is a recent transformer model that takes inputs of a few images to predict 3D Gaussian scenes or objects. We chose this model due to FreeSplatler’s capability to reconstruct accurate 3D models quickly (within approximately 3 seconds) without requiring explicit camera pose information and with only a few frames of input, all of which are limitations in the original 3D Gaussian Splatting. The inclusion of background data in these images enhanced the overall quality of the reconstructed model. We use the Splatviz library to display the 3D result interactively and also save to disk a re-rendering of the 3D result from various angles.

FreeSplatler has two separate models for scene and object reconstruction. We exposed options for using both the scene and object level reconstruction models of FreeSplatler, but found the scene model worked better. The object model required good background removal in the input images, which was not always possible. However, one limitation of the scene model is that it can only take 2 frames as input.

Interestingly, the original codebase for FreeSplatler ran quite slowly on a laptop 4080 RTX GPU. We noticed that after disabling sections of the code that loaded extraneous video generation models, the runtime returned to normal, i.e. the transformer inference takes under 3 seconds.

3 Experiments

For our primary demo which was submitted, we recorded the 5 stage workflow described in the previous section. Our target was a sitting human. The

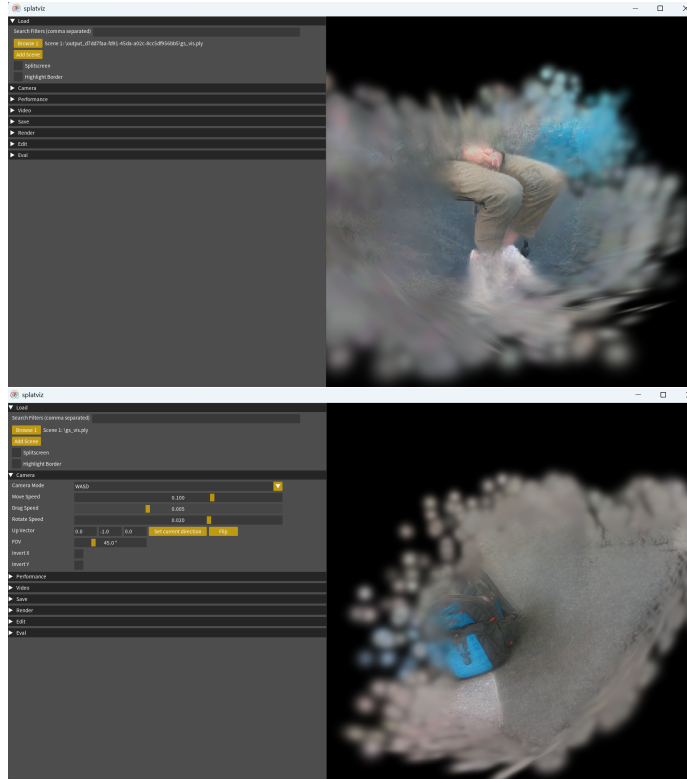


Figure 1: A few example 3D results as seen in the interactive Splatviz renderer.

robot successfully detected and approached the target, and executed the circumnavigation movement with occasionally swiveling to obtain quality photos. The laptop server reconstructed the result quite quickly, and the reconstruction quality based on the Splatviz renderer appears to be quite good.

We list a few strengths that can be noticed in our demo:

Target Flexibility and Lack of April Tags Our robot does not require any april tags to navigate towards objects. Our automatic object localization algorithm enables our robot to detect and navigate to objects in its general FOV, as long as the distinction between foreground and background is clear. We also include visuals of other reconstruction targets to demonstrate the flexibility of our approach.

Correct Camera Positioning The camera mounted on the robot cannot actually turn 90 degrees towards the side, so it is impossible to point the camera at the object which sits in the center of the circular path. Our solution of stopping the robot and swiveling it to point towards the center allows us to take accurate snapshots of the object.

Fast Reconstruction By pruning down the original FreeSplatler code, the runtime on a laptop RTX 4080 GPU is greatly increased, and the transformer



Figure 2: Another target we were able to scan with our robot.

Reconstruction Time	Total Runtime	Images Received	3D File Size
1.4 sec	≤ 1 min	4	27 MB

Table 1: Various quantitative metrics for our robot.

model take around 2 seconds to run.

Interactive 3D Result The 3D Gaussian representation can be rendered in real-time unlike other techniques such as NeRF, allowing users to play with the 3D result on the laptop server in real-time.

4 Challenges and Solutions

We have described some problems and clever design choices we made to address them in the previous sections. Here we disclose some *unexpected* challenges we faced:

Corrupted Environment and Setup Professor Zhi mentioned that the robot had been used previously by a master’s student. We initially had trouble getting the robot to move the wheels or arm, accompanied by several error messages from ROS related to the environment. After a few days of no success, we eventually noticed an expected file was missing, and concluded that the previous user of the robot had corrupted the environment. We followed a fresh install of ROS and the robot software, and successfully fixed the environment. We also had to follow the set-up instructions for the iRobot Create3 base, as it turns out the previous owner of the robot had not properly set this up, or had set it up for the wrong ROS version.

Last Minute Breakdown On the morning of the demo, we had left the robot with the other team sharing with us overnight, but awoke to their message that the robot had broken down. We received the robot before the demo and concluded that there was no way to move any of the robot joints, besides the wheel. However, we quickly refactored our code to avoid robot joint commands, and were able to provide a substitute workflow for our robot relying only on wheel movement to orient the camera and robot.

Robot Movement Issues We noticed our circular robot movement did not always match our desired radius. We ran a sanity check where we tried to make the robot drive straight, and found the robot always veered left for some reason. We compensated by adding a rightward turn in our code to counteract this strange movement.

Networking Networking with the robot was a difficult part of this project. To connect with the robot computer from our laptop, we had to connect both robot and our laptop to the same network and use ssh. This network had to be a phone hotspot, which was problematic since the hotspot often was not strong enough to be found. We found connecting the phone directly to the robot or laptop with a USB improved things. We initially had great trouble moving the robot base and wheels. One teammate suggested we turn on “Maximize compatibility” in the iPhone hotspot, and that worked to bring the base into the network too.

We sent the images from the robot to the laptop server via ZeroMQ. We also had trouble getting this to work initially, but suspecting a firewall issue, we found that the laptop WiFi settings had to be set to “Private Network” to actually make its address visible from the robot.

5 Current Limitations and Future Work

Additional Arm Webcam Originally, we discussed having an additional webcam on the gripper of the arm. This webcam could be extended to capture object interiors, like the inside of a box. We were not able to cover it in the time for this project. We made an effort to experiment with this webcam and successfully used it to replace the original RGBD for capturing, but also found that there were not enough USB ports to support both this webcam and the original RGBD camera. Since the RGBD is more critical for object localization and detection, we did not further pursue this.

Networking Building our current project relied on connection over a smartphone hotspot, which faced reliability issues in different environments. Given more time and resources, we would include exploring alternative networking solutions better suited for robotic applications.

Movement Accuracy We observed consistent veering during what should have been a straight movement, requiring constant adjustments to our code. This might have occurred due to technical reasons or the type of surface on which we experimented. With more time, we would perform a thorough calibration of the movement system and implement ways to ensure a more precise navigation.

Enhanced Object Detection Our current object detection implementation relies on RGB-Depth segmentation, which performs decently when there is a clear distinction between the foreground target object and the background. Future work would incorporate a more robust detection algorithm that can better estimate the position and distance of the target object. It would also have been nice to utilize the LiDAR sensor to provide additional spatial information in more diverse environments.

Adaptive Path Planning The path trajectory we currently set is a fixed circular motion regardless of the shape of the target object and the surrounding environment. A more robust approach would involve real-time mapping of the space and a more adaptive path planning that adjusts the camera position based on the geometry of the object and space available to the robot. SLAM (Simultaneous Localization and Mapping) could play a part in this improvement.

Improved 3D Reconstruction While FreeSplatter provides impressive results given just two images, incorporating a more sophisticated algorithm that can take in more images from different angles would produce even more detailed 3D reconstructions. We mentioned that with better quality captures, we could remove the backgrounds on input images and use the object version of FreeSplatter that takes more than 2 inputs. We could also experiment with InstantSplat, which is more aligned with the original 3D Gaussian Splatting as it is an optimization-based reconstruction algorithm, and has the advantages of requiring few images and no camera poses.

Quantitative Evaluation With our current evaluation being primarily qualitative, the implementation of quantitative metrics to assess the precision of the reconstruction would provide more objective measures of performance.

Conclusion Although our current implementation successfully demonstrates object scanning and 3D reconstruction, numerous opportunities for improvement remain. These enhancements would significantly increase the system’s reliability and improve the overall performance, potentially expanding its applications similar to real-world scenarios.

References

- [1] Florian Barthel et al. *Gaussian Splatting Decoder for 3D-aware Generative Adversarial Networks*. 2024. arXiv: 2404.10625 [cs.CV].
- [2] Bernhard Kerbl et al. “3D Gaussian Splatting for Real-Time Radiance Field Rendering”. In: *ACM Transactions on Graphics* 42.4 (July 2023). URL: <https://repo-sam.inria.fr/fungraph/3d-gaussian-splatting/>.
- [3] Jiale Xu, Shenghua Gao, and Ying Shan. “FreeSplatter: Pose-free Gaussian Splatting for Sparse-view 3D Reconstruction”. In: *arXiv preprint arXiv:2412.09573* (2024).